

LogExpertSolver: A Multi-Agent Framework for Domain-Specialized Log Parsing

Chenxi Mao

Sun Yat-sen University
maochx5@mail2.sysu.edu.cn

Yuxin Su

Sun Yat-sen University
suyx35@mail.sysu.edu.cn

Dan Li

Sun Yat-sen University
lidan263@mail.sysu.edu.cn

Abstract—Log parsing, as a process of extracting structured information from semi-structured raw log data, is a crucial step in log analysis workflows. Rule-based parsing methods often overlook the rich semantic information contained in logs. Recently, LLM-based methods face three major challenges: 1) limited understanding of domain-specific logs due to lack of professional domain knowledge; 2) significant parsing costs and potential data privacy risks associated with commercial LLMs like ChatGPT; and 3) the Group Accuracy (GA) is highly susceptible to individual anomalous data. To address these challenges, we propose LogExpertSolver, a multi-agent framework for domain-specialized log parsing. This framework effectively reduces manual intervention in log parsing through inter-agent collaboration mechanisms. LogExpertSolver employs locally deployed medium-scale language models to construct multiple agents with diverse domain expertise. By decomposing complex log parsing tasks into a series of fine-grained subtasks, which are handled by corresponding expert agents, the framework achieves efficient log parsing. Through evaluation on the LogHub2.0 dataset, LogExpertSolver achieves an average Parsing Accuracy (PA) of 0.915, surpassing state-of-the-art parsers (LibreLog and LILAC) by 6.2% and 7.3% respectively.

Index Terms—Log Parsing, Large Language Model, Multi-Agent

I. INTRODUCTION

Logs provide crucial runtime information for software system debugging, monitoring, and maintenance [1]. However, with the continuous expansion of software system scales, log data exhibits characteristics of massive volume and high complexity, posing significant challenges to manual log analysis methods [2]. To improve the efficiency of log analysis, researchers have proposed numerous automated solutions for various log-related tasks, including anomaly detection [3]–[7], log compression [8]–[10], and root cause analysis [11], [12]. Among these log analysis tasks, log parsing is typically regarded as the foundational step of the entire pipeline.

Log parsing aims to extract structured information from semi-structured raw log data, converting raw log messages into standardized log templates [13]. Each log message is generated by corresponding log statements in the source code. The Headers are uniformly standardized by the logging framework adopted by the project, typically containing metadata information such as timestamps, log levels. The Content section of log messages primarily consists of two core components: 1) static text strings that describe system events without any variable parameters; 2) dynamic parameters that change in real-time

during system operation, used to record specific contextual information and state data.

Although some studies have previously proposed various log parsers [13]–[16], efficient and accurate log parsing still faces many challenges [2]. Huo et al. [16] pointed out that traditional methods are primarily based on syntactic rules, ignoring the rich semantic information contained in log content. Such syntax-based parsing methods typically have limitations in parameter identification [17], [18]. Addressing this issue, Ma et al. [2] proposed: log messages are essentially a hybrid of natural language descriptions and code-like variables, and it can be viewed as a "translation" task from raw logs to structured templates. Based on LLMs' advantages in semantic understanding, recent researchers have begun to focus on semantic information mining in logs, achieving significant progress in semi-structured log data parsing [13].

However, different log data typically represent runtime information from components in different domains, such as components responsible for databases and network connections, which differ in semantics and formats, and their characteristics may be significantly different. Therefore, approaches that heavily depend on predefined rules may lack flexibility, are constrained to specific systems, and fail to handle complex and variable log formats [15]. When directly using general-purpose LLMs for parsing, due to the lack of domain-specific professional knowledge, they often cannot meet the requirements for parsing accuracy and domain adaptability in practical applications.

Furthermore, many current LLM-based log parsing solutions rely on commercial large models such as ChatGPT [19], [20], where such commercial API services typically charge high fees. Considering the large-scale batch processing characteristics of log parsing tasks, the contradiction between massive log data and LLMs' limited context windows leads to significant computational overhead, including time costs and economic costs. Large-scale parameter computation not only reduces the overall efficiency of log parsing but also causes token consumption to grow linearly with the number of logs, making parsing costs rise rapidly. Therefore, reasonable control of LLM invocation costs is of great significance for practical deployment applications [14]. Efficient log parsing systems should be able to meet large-scale batch processing requirements while maintaining low resource consumption. Additionally, using commercial models poses potential data

privacy risks, as log data typically contains sensitive information and business data from software system runtime [21].

Moreover, many log parsers adopt clustering-based template extraction strategies [19], [21], using GA as an important performance evaluation metric. However, recent research [2] has discovered inherent flaws in this approach: in experiments on the Spark dataset, although PA reached 0.9995, just one log parsing error caused 300 logs with the same template to be incorrectly divided into two different clustering groups, causing the GA metric to plummet to 0.85. This phenomenon reveals the high sensitivity of clustering-based log parsing methods and GA metric to individual parsing error cases.

To address the above challenges, this paper proposes LogExpertSolver, a multi-agent log parsing framework that integrates domain expert knowledge. The framework fully leverages the inherent language understanding and logical reasoning capabilities of large language models [22], achieving automation of the log parsing process and reducing the cost of manual intervention. To balance parsing performance with resource consumption, LogExpertSolver uses locally deployable medium-scale open-source large models (such as Qwen3-14B) to construct the multi-agent framework. Compared to large-scale commercial models, open-source models with moderate parameter counts not only reduce parsing time overhead, but local deployment also enhances data privacy protection capabilities and reduces token consumption costs.

The main contributions of this paper are as follows:

- LogExpertSolver first combines domain expert knowledge with multi-agent collaboration mechanisms and overcomes the limitations of existing syntax-based and LLM-based log parsers.
- To reduce manual intervention, LogExpertSolver designs task decomposition mechanisms, achieving full automation of the parsing process through multi-agent collaborative processing.
- To reduce parsing costs while meeting large-scale batch processing and low resource consumption requirements, we adopts locally deployed medium-scale open-source LLMs to replace expensive commercial LLMs.
- Comparative experiments were conducted on 14 different datasets contained in LogHub2.0. LogExpertSolver demonstrates 6.2% and 7.3% higher PA compared to state-of-the-art parsers.

II. BACKGROUND

A. Large Language Model

Large Language Models (LLMs), as prominent representatives of deep neural network technologies, are built upon the Transformer architecture and demonstrate exceptional language understanding and generation capabilities through pre-training on large-scale, diverse text corpora.

With the continuous expansion of model scales and the increasing diversity of training corpora, In-Context Learning enables LLMs to achieve rapid task adaptation and effective generalization based on limited demonstration samples [2].

In log parsing scenarios, this characteristic helps models extract key information from diverse log formats, including dynamic variable identification, structured feature extraction, and semantic pattern understanding.

B. Multi-agent Systems

Independent LLMs inherently possess the basic characteristics of simple agents since they can receive text inputs and generate corresponding text outputs, making the boundary between LLMs and LLM-based agents relatively blurred [23].

Compared to single LLM-based agent systems, multi-agent systems provide enhanced capabilities [24]: 1) specializing general-purpose LLMs into specialized agents with different capabilities, achieving functional division; 2) establishing effective interaction mechanisms among agents, enabling more realistic simulation of collaborative patterns in complex real-world environments. The dynamic mechanism combining collaboration and competition enables multi-agent systems to possess stronger adaptability and problem-solving capabilities.

III. METHOD

A. Overview of LogExpertSolver

The LogExpertSolver method proposed in this paper constructs expert agents with specialized knowledge in different log domains, enabling automatic extraction of domain classification lists from source code and identification of the domain affiliation for each log entry, thereby achieving high-quality log parsing without the need for targeted model fine-tuning.

As shown in Figure 1, the LogExpertSolver framework consists of four phases: Domain Component Extraction Phase, Domain Filtering and Optimization Phase, Log Domain Classification Phase, and Log Processing and Standardization Phase.

LogExpertSolver is executed by a specific group of specialized agents. In total, the framework defines six core agents: (1) a Domain Extraction Expert, (2) a Domain Filtering and Optimization Expert, (3) a Log Domain Classification Expert, (4) a Log Preprocessing Expert, and (5) two Log Standardization Experts (text and numerical). These agents communicate and coordinate through the CrewAI, which manages task delegation, result passing, and workflow sequencing via JSON-based message exchange. Each agent receives structured task instructions, processes data independently, and outputs results in standardized JSON format, enabling seamless inter-agent communication.

In terms of technical implementation, LogExpertSolver adopts Qwen3 as the foundational LLM. We selected the relatively smaller instruction-tuned model Qwen3-14B as the experimental foundation.

CrewAI Framework Integration. CrewAI is an open-source multi-agent coordination framework that provides task scheduling, role definition, and inter-agent communication. It enables multiple specialized agents to collaborate efficiently within a unified workflow. In LogExpertSolver, CrewAI acts as the coordination backbone: the framework manages the initialization of expert agents, assigns specific preprocessing and standardization tasks, and synchronizes data exchange

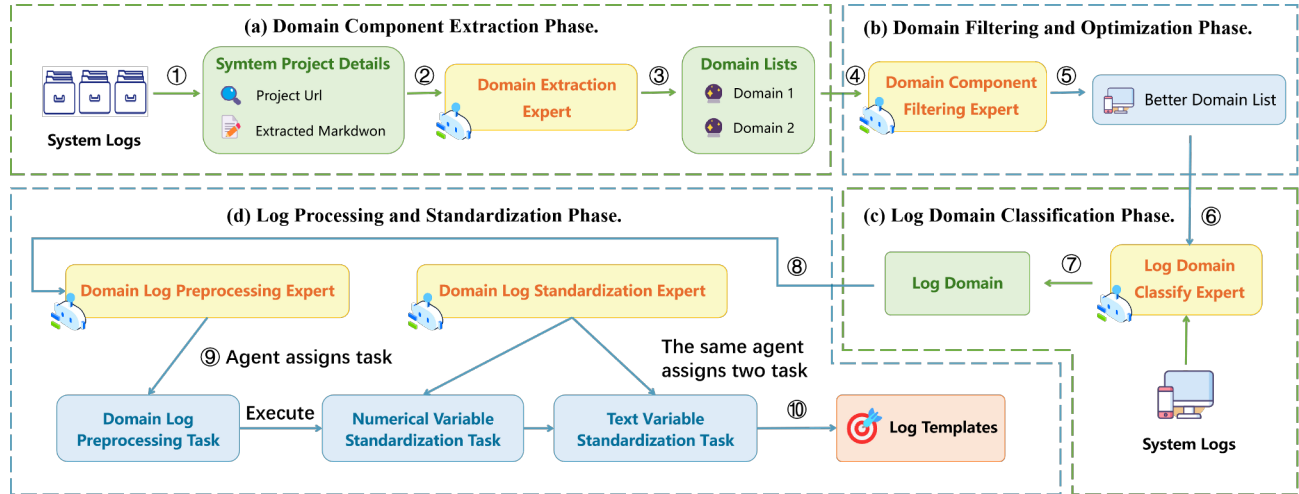


Fig. 1. Overview of LogExpertSolver.

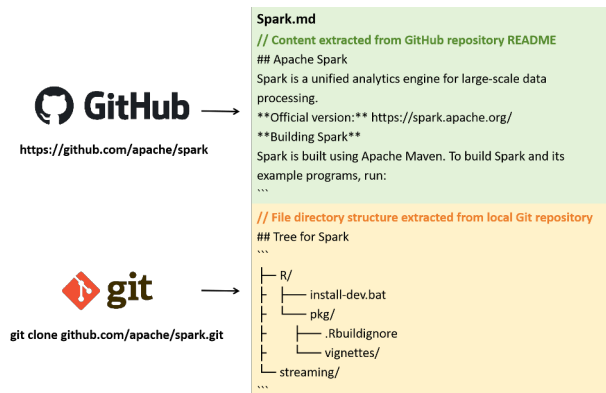


Fig. 2. Example markdown input for Expert processing.

between them. This design ensures that each agent operates autonomously while maintaining consistent workflow execution across the system.

B. Domain Component Extraction Phase

The first core phase is the Domain Component Extraction Phase, which comprises three key steps. First, LogExpertSolver traverses the candidate logs, extracting the System Details. For acquiring metadata information of log projects, the system provides two distinct input processing approaches.

Approach 1: Direct URL Input Processing. The approach involves directly inputting the project URL to the LLM, enabling it to autonomously read and analyze webpage content to extract project information. Through real-time access and analysis of external web resources, LLMs can obtain richer and more up-to-date project information [25]. However, due to the diversity, dynamicity, and structural complexity of website content, as well as the strict limitations imposed by token constraints on LLM output capabilities [26], LLM cannot perform

complex webpage interactions and deep content parsing like real browsers.

Approach 2: Markdown Format Conversion Processing. The approach adopts a more LLM-friendly Markdown format for information input. For projects that only have textual descriptions without source code, LogExpertSolver uses the Markdowner to extract webpage content from URLs and convert it into structured Markdown format, thereby improving LLM's understanding.

For project logs with clearly available source code, LogExpertSolver employs a more in-depth strategy: the system first downloads the complete Git repository of the project, then uses the git2md to extract README file and directory tree structure, integrating this information into a Markdown file. As shown in Figure 2, this example demonstrates the process of inputting a project Markdown file with clearly available source code to agents for domain extraction. Taking the Spark as an example, the generated Spark.md file contains description information extracted from the GitHub homepage in the first half, while the second half includes hierarchical directory structure extracted from the local Git repository.

Compared to the direct URL input approach, while the Markdown file extraction method shows improvements in information processing efficiency, it may result in the loss of information. Particularly due to the limitations of LLM input context length, the tree-like file structure shown in Figure 2 requires appropriate filtering and compression processing to ensure that key information can be effectively conveyed within the limited context window.

Next, the system inputs the original URLs and Markdown files containing project information into the Domain Extraction Expert for processing. Considering that extracting domain lists from source code is a critical and complex task requiring deep code comprehension capabilities, unlike other expert modules in the framework that use Qwen3, the Domain Extraction Expert employs Claude-3.5-Sonnet as its foundational model.

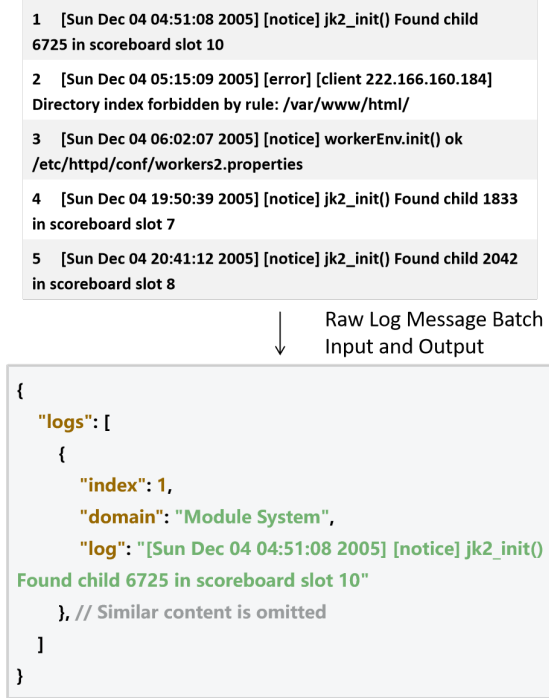


Fig. 3. Example domain output.

Claude-3.5-Sonnet demonstrates significant advantages in handling complex codebase analysis and advanced tool usage in programming-related tasks [27], enabling more accurate understanding of code structure and project architecture, thereby providing more precise domain identification capabilities.

Finally, the expert generates corresponding domain lists for each project, which contain two types of results extracted from different information sources.

C. Domain Filtering and Optimization Phase

LogExpertSolver inputs the two domain list options into the Domain Component Filtering Expert. By designing prompts that meet specific scenario requirements, this expert can comprehensively evaluate the applicability of the two candidate lists, ultimately selecting Better Domain List to serve as the standard classification basis for subsequent log classification and processing tasks.

Taking the Zookeeper as an example, the system inputs two candidate lists: the list extracted from URL "[Server', 'Client', 'ZooKeeper Session', 'Quorum', 'Storage']" and the list extracted from Markdown files "[Server', 'Client', 'Quorum', 'Persistence', 'Watch']". After analysis by the Domain Component Filtering Expert, Claude-3.5-Sonnet ultimately selected the domain list extracted from the Markdown file "[Server', 'Client', 'Quorum', 'Persistence', 'Watch']" as the basis.

D. Log Domain Classification Phase

LogExpertSolver uses the Better Domain List as the classification standard, inputting it together with system log into

the Log Domain Classification Expert, which executes the log domain classification task. Figure 3 demonstrates an example of the expert performing log classification.

To improve classification processing efficiency, LogExpertSolver adopts a batch processing strategy, setting the batch size to 5 logs. The system inputs every 5 logs as a batch to the classification expert, and the expert module returns structured JSON format output after processing. The JSON output contains 5 list items, with each list item corresponding to one log record in the batch, specifically including three key fields: log index ("index"), the component domain to which the log belongs ("domain"), and the original log content ("log").

E. Log Processing and Standardization Phase

In the Log Processing and Standardization Phase, LogExpertSolver performs domain-aware log parsing and normalization. This phase consists of three main steps: domain-specific preprocessing, variable standardization, and template generation.

To achieve this, the system configures two expert agents based on the CrewAI collaborative framework: a Log Preprocessing Expert and a Log Standardization Expert. Each expert focuses on distinct but complementary tasks. The Preprocessing Expert extracts key message content from raw logs, while the Standardization Expert normalizes textual and numerical variables to produce unified templates.

This design allows LogExpertSolver to leverage domain knowledge effectively, ensuring structured and consistent parsing results. The entire workflow operates through CrewAI's multi-agent coordination mechanism, where each agent executes its domain-specific tasks defined by role-based prompts and standardized task schemas. Each prompt defines the agents' expertise and expected JSON-formatted output, enabling reliable downstream integration.

Following the domain classification process (Figure 3), each log is assigned to a corresponding functional domain and then passed to the two specialized agents for further processing. For instance, given the log message Dec 10 07:07:45 LabSZ sshd[24206]: Failed password for invalid user test9 from 52.80.34.196 port 36060 ssh2, the system determines that it belongs to the Authentication domain. Figure 4 shows the complete processing pipeline for this entry.

First, the Domain Log Preprocessing Expert extracts the core content Failed password for invalid user test9 from 52.80.34.196 port 36060 ssh2. Then, the Domain Log Standardization Expert replaces numeric and textual variables with placeholders, transforming it into a standardized log template. This phase concludes the parsing process and provides structured log templates for subsequent analysis tasks.

IV. IMPLEMENTATION

A. CrewAI Framework Overview

CrewAI is a lightweight multi-agent orchestration framework that enables large language model (LLM) agents to collaborate through structured workflows. It comprises three core components:

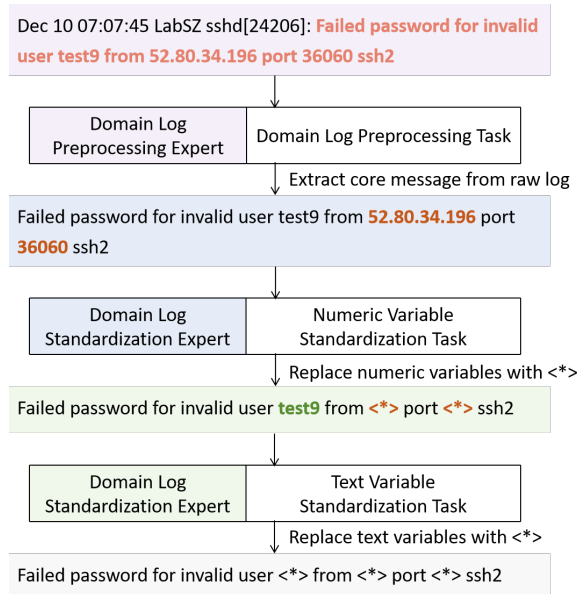


Fig. 4. Concrete Log Message Example

- **agent.py**: defines intelligent agents with specific roles, skills, and domain expertise;
- **task.py**: specifies task descriptions, expected outputs, and links to corresponding agents;
- **crew.py**: manages orchestration, coordinating multiple agents and tasks for sequential or parallel execution.

This modular design allows LogExpertSolver to separate domain expertise (in agents) from task logic (in workflows), improving clarity, extensibility, and reproducibility.

B. Implementation of the Log Processing and Standardization Phase

In the CrewAI-based implementation of LogExpertSolver, specific log parsing operations are executed through well-defined agenttask structures across the above three modules.

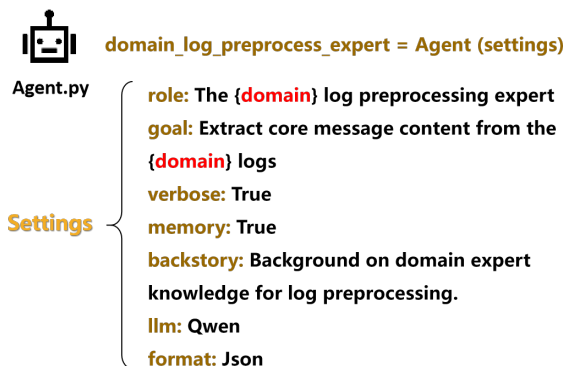


Fig. 5. Agent.py Example

- **Agent Definition**: Figure 5 demonstrates the definition of a domain-specific Log Preprocessing Expert in *agent.py*.

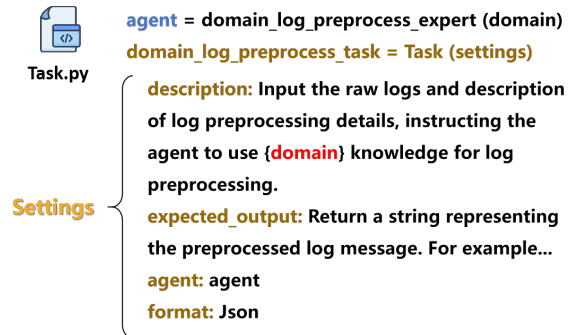


Fig. 6. Task.py Example

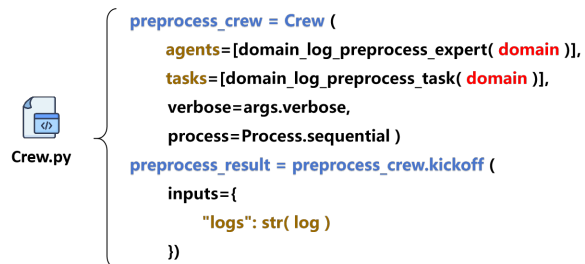


Fig. 7. Crew.py Example

An *Agent* object is created, and its role is set as a preprocessing expert equipped with domain knowledge. The system defines domain-specific log preprocessing expertise through carefully designed prompts, explicitly specifying the use of Qwen3 as the foundational model and JSON as the standardized output format. This ensures structured and consistent extraction of key log content.

- **Task Configuration**: Figure 6 illustrates how *task.py* configures preprocessing tasks. A *Task* object named *domain_log_preprocess_task* is created, whose *description* field details the tasks purpose receiving raw log inputs and guiding the preprocessing expert to extract core content. The *expected_output* field defines the required output schema and provides an example. The *agent* field binds the task to the instantiated expert object, ensuring that the models output remains structured in JSON format.
- **Crew Coordination**: Finally, the *crew.py* module coordinates all agents and tasks. Figure 7 shows how the *Crew* class is instantiated as *preprocess_crew*, registering the agents and tasks defined above. The execution mode is set to sequential, and the system calls the *kickoff()* function to process input logs and produce final results.
- **Example of End-to-End Processing**: Figure 4 demonstrates the complete end-to-end process for a concrete log message. The raw log is first preprocessed by the domain expert, then passed through two standardization subtasks the Text Variable Standardization Task and the Numerical Variable Standardization Task. Finally, a standardized log template is generated, completing the entire log parsing

and normalization process.

V. EVALUATION

A. Dataset

To comprehensively evaluate our proposed LogExpertSolver method, we first need to select appropriate benchmark datasets. Based on the raw log data and ground truth provided by the selected dataset, we will conduct experimental evaluations from three dimensions: the overall effectiveness of our method, the contribution of functional modules, and the impact of various base LLMs on system performance. When selecting benchmark datasets, we focus on two key factors: the dataset should contain diverse log data from real-world systems and provide accurate templates as evaluation ground truth. After in-depth analysis of existing datasets, we chose LogHub2.0 as our experimental dataset for the following reasons.

Zhu et al. [28] constructed a log parsing benchmark dataset called LogHub, which contains real log data from 19 different software systems. It has been widely used for evaluating and comparing the accuracy of log parsers [29]–[31]. However, subsequent studies [17] discovered instances of incorrectly labeled log templates, which affected the reliability of benchmark testing.

LogHub2.0 [32] is the latest large-scale log parsing annotation dataset constructed by the LogPAI team based on LogHub, collecting high-quality real datasets from 14 software systems. Consistent with LogHub, each system similarly contains 2,000 log messages along with their corresponding log templates. Currently, LogHub2.0 has been widely adopted in related research on log parser evaluation and comparison [19], [33].

Given that LogHub2.0, as an updated version, possesses higher data quality, while the annotation errors in some logs of the original LogHub dataset require substantial manual costs for correction, this paper selects LogHub2.0 as the benchmark testing dataset for experiments in order to obtain more reliable benchmark testing results and minimize manual intervention in the testing process to the greatest extent possible.

It is worth noting that the Loghub2.0 dataset is derived from operational logs of real-world systems such as Hadoop, HDFS, and Spark, thus sufficiently capturing the characteristics of practical log parsing scenarios.

B. RQ1: What is the effectiveness of LogExpertSolver?

Motivation. Accuracy is the most crucial metric for evaluating the effectiveness of log parsers, as it significantly impacts downstream tasks [21]. In RQ1, we investigate the effectiveness of LogExpertSolver.

Method. We compare LogExpertSolver with other parsers. AEL [34], Drain [35], and UniParser [30] perform log parsing based on syntax and semantics, demonstrating superior performance among traditional log parsers. LogPPT [13], LLM-Parser [2], and LibreLog [21] are recently proposed LLM-based parsers that offer enhanced semantic understanding and higher PA.

Result. As shown in Table I, LogExpertSolver achieves the highest PA on most systems. It achieves an average PA of 0.915, outperforming other parsers.

T5Base and LILAC show relatively weaker performance among LLM-based parsers. Compared to LogExpertSolver, T5Base and LILAC exhibit notable performance gaps. While these parsers all use demonstrations to help language models understand the log parsing process, the key difference lies in how expert knowledge is applied. LILAC relies on contextual learning for demonstration, while T5-base uses few-shot fine-tuning, showing limited capability in capturing complex log patterns, as evidenced by their poor performance on systems like BGL and Spark. In contrast, LogExpertSolver incorporates domain parsing rules and thinking patterns through a multi-agent expert system, enabling it to analyze and understand logs like professionals. The effectiveness of this strategy is reflected in parsing results across various systems, achieving extremely high accuracy in different types of system logs such as Apache, Zookeeper, and BGL. This indicates that systematic incorporation of expert knowledge is more crucial than mere example demonstrations in log parsing tasks.

Notably, despite LILAC's use of the powerful commercial model ChatGPT as its foundation model, its performance still falls behind LogExpertSolver, which uses locally deployed smaller models, on systems like Hadoop, Zookeeper, and BGL. According to Ma et al. [21], when LILAC replaces ChatGPT with smaller open-source models, its parsing capability significantly decreases, and LogExpertSolver surpasses LILAC to achieve the highest PA in systems like Spark. This not only demonstrates the crucial role of expert knowledge in log parsing but also shows that well-designed agent systems can effectively compensate for capability gaps due to model size limitations, enabling even smaller models to achieve efficient and accurate log parsing. This ability to maintain high performance despite model size constraints gives LogExpertSolver stronger deployment advantages and broader applicability in practical applications.

C. RQ2: How does different settings impact the result of LogExpertSolver?

Motivation. LogExpertSolver implements multi-step processing, we conduct a comparative experiment to evaluate the impact of different subtask configurations and demonstrate the effectiveness of the approach.

Method. We replace the log processing and standardization phase with a clustering-based template extraction scheme. This comparative approach uses the Drain algorithm to cluster each preprocessed log online, obtains the cluster for that log, and then inputs the logs from that cluster to a domain expert for template extraction. This experiment aims to compare our fine-grained multi-step processing workflow with traditional clustering methods. The results are shown in Table II.

Result. (1) Overall Performance Comparison. In terms of average PA, LogExpertSolver's complete workflow achieves an average score of 0.915, while the clustering-based Cluster

TABLE I
A COMPARISON OF THE PARSING ACCURACY (PA) FOR THE STATE-OF-THE-ART PARSERS AND LOGEXPERTSOLVER.

	AEL	Drain	UniParser	LogPPT	LILAC	LLMParser-T5Base	LibreLog	LogExpertSolver
Hadoop	0.535	0.541	0.889	0.666	0.832	0.837	0.870	0.989
HDFS	0.621	0.621	0.948	0.943	0.999	0.966	1.000	0.783
OpenStack	0.029	0.029	0.516	0.406	1.000	0.941	0.830	0.536
Spark	–	0.394	0.795	0.952	0.973	0.423	0.888	0.943
Zookeeper	0.842	0.843	0.988	0.845	0.687	0.840	0.849	1.000
BGL	0.406	0.407	0.949	0.938	0.958	0.244	0.929	0.990
HPC	0.741	0.721	0.941	0.997	0.705	0.707	0.973	0.992
Thunderbird	0.163	0.216	0.654	0.401	0.559	0.347	0.694	0.975
Linux	0.082	0.111	0.164	0.168	0.765	0.880	0.901	0.667
Mac	0.245	0.357	0.688	0.390	0.638	0.613	0.653	0.955
Apache	0.727	0.727	0.942	0.948	0.996	0.990	0.996	1.000
OpenSSH	0.364	0.586	0.289	0.654	0.941	0.781	0.495	0.986
HealthAPP	0.311	0.312	0.817	0.997	0.729	0.967	0.973	0.990
Proxifier	0.677	0.688	0.634	1.000	1.000	0.981	0.897	1.000
Average	0.442	0.468	0.730	0.736	0.842	0.751	0.853	0.915

TABLE II
LOGEXPERTSOLVER PERFORMANCE UNDER DIFFERENT SETTINGS.

	Average	Hadoop	HDFS	OpenStack	Spark	Zookeeper	BGL	HPC	Thunderbird	Linux	Mac	Apache	OpenSSH	HealthAPP	Proxifier
LogExpertSolver	0.915	0.989	0.783	0.536	0.943	1.000	0.990	0.992	0.975	0.667	0.955	1.000	0.986	0.990	1.000
Cluster	0.866	0.820	0.807	0.718	0.942	0.978	0.913	0.717	0.946	0.832	0.872	0.999	0.978	0.973	0.638

approach scores 0.866. Among the 14 systems, LogExpertSolver demonstrates better or comparable performance in 11 systems, indicating that this framework has certain applicability in most systems.

The performance difference between the two methods is particularly notable in some datasets. For instance, in the HPC dataset, LogExpertSolver achieves an accuracy of 0.992, while the Cluster achieves 0.717. Similar gaps are also observed in Proxifier datasets. These performance differences may reflect the inherent mechanical differences between the two technical approaches. The clustering-based approach’s final effectiveness partly depends on the accuracy of the upstream clustering algorithm. If multiple templates are mixed within a log cluster, subsequent template extraction may be affected, leading to errors. LogExpertSolver’s design does not directly rely on clustering, instead performing independent variable extraction and processing for each log entry, which may help avoid potential problems caused by inaccurate clustering when handling more complex log formats.

(2) Superior Performance of Cluster Approach in Certain Systems. In HDFS, OpenStack, and Linux datasets, the Cluster approach achieves higher PA. One possible explanation is that these datasets contain fewer contents and have more regular formats, allowing the Drain algorithm to effectively and accurately group logs, and a single template extraction Agent is sufficient to complete the extraction task. In such cases, the direct process of clustering and template extraction might be a straightforward and effective method.

Overall, the proposed log processing and standardization phase is an effective component in the LogExpertSolver. Compared to the traditional process of clustering followed by template extraction, fine-grained processing of individual log

entries can bring performance improvements across multiple datasets, especially in scenarios with richer log content and more diverse formats.

D. RQ3: What is the effectiveness of LogExpertSolver with different LLMs?

Motivation. Unlike log parsers such as LILAC [13] that use commercial LLMs, LogExpertSolver employs open-source, smaller LLMs to mitigate parsing costs. Different LLMs exhibit varying capabilities due to their unique architectures and pre-training data [21]. In RQ3, we evaluate LogExpertSolver’s performance across various open-source local LLMs.

Method. In addition to Qwen3-14B, we selected two other models with similar parameter sizes to compare the performance of log parsing using different LLMs. These models include Qwen2.5-14B and DeepSeek-R1-14B, which are commonly used in research and practice.

Result. Table III shows the parsing performance of LogExpertSolver using different LLMs. Among all models, Qwen3-14B achieved the best overall performance. As shown in Table III, Qwen3-14B achieved the best performance on 9 out of 14 datasets, with an average accuracy of 0.915. Notably, it achieved perfect PA 1.000 on Zookeeper, Apache, and Proxifier datasets. Qwen2.5-14B, as the second-best model, achieved an PA of 0.888 and performed best on 4 datasets: OpenStack, Thunderbird, Linux, and HealthAPP. It also achieved perfect accuracy on the Apache dataset. However, the model performed poorly on the Proxifier dataset with an accuracy of only 0.451, significantly lowering its overall performance. DeepSeek-R1-14B showed relatively weaker overall performance with an average accuracy of 0.718. Although it achieved the best score 0.791 on the HDFS dataset, its

performance on most datasets was inferior to the other two models.

These performance differences can be explained by the models' characteristics and design objectives. The Qwen series models were exposed to a large amount of Chinese and English technical documentation, code, and system log data during pre-training, making them particularly sensitive to structured text and system log formats. Specifically, Qwen3-14B employs more advanced instruction fine-tuning techniques, enhancing its instruction-following capability and contextual understanding, which proves particularly important when parsing logs from complex systems. In contrast, the DeepSeek series was primarily optimized for reasoning and mathematical computation, which may explain its relatively weaker performance in text pattern recognition tasks. This is validated in the HDFS and Apache datasets, where the standardized log formats and strong logical reasoning requirements match DeepSeek's strengths, enabling it to achieve good results.

Looking at cross-dataset performance, all models generally perform well on logs with uniform formats and simple structures (like Apache) but struggle with logs having diverse formats and complex structures (like Linux). This phenomenon reflects the models' varying adaptability when handling logs of different complexity levels. Particularly noteworthy is Qwen2.5-14B's abnormally low score 0.451 on the Proxifier, which may be due to significant differences between the dataset's log format and the model's pre-training data, or systematic bias in handling certain specific log patterns. In the Proxifier, logs contain formats like 'proxy.sse.sysu.edu.cn:5070', where the model struggles to identify and process the proxy server hostname 'proxy.sse.sysu.edu.cn' as a generic placeholder, resulting in incorrect log templates like 'proxy.sse.sysu.edu.cn:<*>'.

These findings provide important insights for future model improvements. First, increasing the diversity of log data during pre-training may help enhance model generalization capability. Second, specialized fine-tuning for specific types of log formats might improve model performance in corresponding scenarios. Finally, designing dedicated attention mechanisms for structured text parsing tasks is a direction worth exploring. Overall, these results indicate that model performance in log parsing tasks is closely related to pre-training data, architectural design, and optimization objectives, and future improvements should consider these aspects comprehensively.

VI. RELATED WORK

A. Log Parsing

Based on parsing algorithms, log parsing methods can be categorized into types: unsupervised methods, supervised methods, and LLM-based methods.

1) *Unsupervised Log Parsing Parsers*: Unsupervised log parsers utilize hand-crafted features to extract log templates, primarily based on three approaches: frequent pattern mining, clustering, and heuristic methods [15].

Frequent Pattern Mining. Identifies repetitive patterns in log messages as log templates. SLCT [36] was the first to

TABLE III
PARSING PERFORMANCE OF LOGEXPERTSOLVER USING DIFFERENT LLMs.

	Qwen3-14B	Qwen2.5-14B	DeepSeek-R1-14B
Hadoop	0.989	0.919	0.500
HDFS	0.783	0.581	0.791
OpenStack	0.536	0.846	0.437
Spark	0.943	0.895	0.611
Zookeeper	1.000	0.989	0.893
BGL	0.990	0.944	0.809
HPC	0.992	0.988	0.597
Thunderbird	0.975	0.977	0.857
Linux	0.667	0.728	0.588
Mac	0.955	0.896	0.801
Apache	1.000	1.000	0.931
OpenSSH	0.986	0.899	0.665
HealthAPP	0.990	0.993	0.793
Proxifier	1.000	0.451	0.794
Average	0.915	0.888	0.718

apply frequent pattern mining to log parsing; LogCluster [37], as an extended version of SLCT, exhibits strong robustness to token position variations.

Clustering Methods. Templates and variables are extracted by comparing common and different parts within groups. Offline modes such as LKE [38], LogSig [39], and LogMine [40] perform batch clustering; online modes such as SHISO [41] and LenMa [42] process new logs in real-time using streaming approaches.

Heuristic Methods. Leverage structural characteristics of log messages, encoding rules for template extraction. Typical methods include parsing tree structures that guide the log matching process by constructing fixed-depth parsing trees [2]. Drain [35] constructs fixed-depth parsing trees to efficiently organize log patterns, achieving SOTA performance among traditional unsupervised parsers [15]. Improvements based on Drain include: POP [43] enables distributed deployment, and SPINE [31] introduces human feedback mechanisms.

2) *Supervised Log Parsing Parsers*: Supervised log parsers extract log templates by training deep learning models on annotated datasets. Traditional unsupervised parsers primarily rely on syntactic analysis, treating log messages as strings and using statistical features to extract common parts, while ignoring semantic information contained in parameters and templates [13], [16], [30]. To address this issue, researchers have proposed supervised methods to mine semantic information from logs. SemParser [16], as the first semantic-based parser, breaks through the key bottleneck of log semantic mining; UniParser [30] effectively captures common log behaviors in heterogeneous log data by learning patterns of log tokens and their neighboring contexts.

3) *LLM-based Log Parsing Parsers*: Deep learning-based log parsers have validated the feasibility and effectiveness of utilizing pre-trained models for log parsing [15].

Early work LogPPT [13] employed prompt-based few-shot learning to fine-tune RoBERTa [44] for capturing log patterns. As a preliminary attempt to apply LLMs to log parsing, it demonstrated higher accuracy than traditional syntax-based

parsers [21]. Subsequent research has continuously optimized methods and performance: Ma et al. [2] compared four different LLMs and proposed LLMParse based on few-shot tuning; Yu et al. [45] proposed LogGenius, an unsupervised framework with zero-shot prompting; Jiang et al. [19] designed the LILAC combining LLMs with adaptive caching mechanisms, reducing computational overhead while improving accuracy.

Recent work has further explored training-free parsing solutions: Ma et al. [21] proposed the LibreLog, achieving unsupervised parsing through fixed-depth grouping trees, retrieval-augmented generation, self-reflection; DivLog [15] utilizes diversity sampling and example selection for training-free parsing; LogBatcher [14] combines clustering partitioning, cache matching, and batch processing mechanisms for training-free parsing; Lemur [46] improves GA by merging similar templates through LLMs.

Unlike existing approaches, LogExpertSolver reduces manual intervention, requiring no heuristic rules, hand-crafted features, or fine-tuning on specific training data. This method is based on a Multi-Agent framework, reasoning capabilities to directly mine semantic information from target logs and parse log templates.

VII. THREATS TO VALIDITY

External Validity. We discovered issues in the truth of certain logs within the LogHub-2.0 dataset. For instance, in the BGL system, we found missing template symbols: the 10th log template "ciod: failed to read message prefix on control stream (CioStream socket to <*>:<*>" lacks a closing ")" symbol, which LogExpertSolver correctly preserved. Similar issues were observed in Mac system logs: for log entry 1875, "vmware-usbarb-25037.log", the truth parses it as "vmware-usbarb-<*>.log", while LogExpertSolver treats it as a complete variable and replaces it entirely with a placeholder; in log entry 1937, for "bssid 5c:50:15:36:bc:03", the truth provides a difficult-to-understand template "bssid <*>:<*>:<*>:<*>", whereas LogExpertSolver more reasonably parses it as "bssid <*>". These issues not only affect the accuracy of performance evaluation but also increase the manual effort required for result verification.

Internal Validity. Logs still retain natural language properties, which can lead LLMs to produce different yet reasonable parsing results for the same log entry. An example is the timestamp handling in log entry 521 of the Mac project, "RTC: Maintenance 2017/7/3 21:47:58": the truth is "RTC: Maintenance <*>/<*>/<*> <*>:<*>:<*>", replacing each number with individual placeholders, while LogExpertSolver treats the entire timestamp as a semantic unit, generating a more concise template "RTC: Maintenance <*>". These different interpretations of complex log structures reflect the subjective nature of log parsing tasks and pose challenges for accuracy evaluation.

To ensure experimental reproducibility and result stability, we implemented several technical measures. Specifically, we set LogExpertSolver's model generation temperature to 0 to eliminate randomness, ensuring consistent parsing results

under identical input conditions. While this setting might limit the model's creativity, it is appropriate for log parsing tasks that require high consistency.

To further ensure reproducibility and transparency, all prompt templates and configuration files used in LogExpertSolver have been released in a public GitHub repository: <https://github.com/maochenxi/logexpertsolver-prompts>.

Construct Validity. As LogExpertSolver relies on project metadata (e.g., GitHub README files) for domain knowledge extraction, inaccuracies or incompleteness in such metadata may affect domain classification and parsing accuracy. Future work will explore adaptive strategies to mitigate this dependency.

VIII. CONCLUSION

In this paper, we introduce LogExpertSolver, a multi-agent framework for domain-specialized log parsing, which combines domain expert knowledge with multi-agent collaboration mechanisms to reduce manual intervention in log parsing. Our comprehensive evaluation on LogHub2.0 demonstrates that LogExpertSolver achieves an average PA of 0.915, outperforming state-of-the-art log parsers (LibreLog [21] and LILAC [19]) by 6.2% and 7.3% respectively. Furthermore, LogExpertSolver addresses the high parsing cost issue associated with commercial large-scale LLMs, meeting the requirements for large-scale and resource-efficient log parsing in practical applications.

ACKNOWLEDGMENTS

The work was supported by the National Natural Science Foundation of China (No. 62202511) and CCF - Sangfor Yuanwang Research Fund.

REFERENCES

- [1] A. Mastropaolo, L. Pascarella, and G. Bavota, "Using deep learning to generate complete log statements," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2279–2290.
- [2] Z. Ma, A. R. Chen, D. J. Kim, T.-H. Chen, and S. Wang, "Llmparser: An exploratory study on using large language models for log parsing," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [3] H. Guo, J. Yang, J. Liu, J. Bai, B. Wang, Z. Li, T. Zheng, B. Zhang, J. Peng, and Q. Tian, "Logformer: A pre-train and tuning pipeline for log anomaly detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 1, 2024, pp. 135–143.
- [4] Y. Huo, Y. Li, Y. Su, P. He, Z. Xie, and M. R. Lyu, "Autolog: A log sequence synthesis framework for anomaly detection," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 497–509.
- [5] Y. Shavit, K. Razmadze, G. Mataev, H. Shteingart, E. Zahavi, and Z. Binshtock, "Semantilog: Log-based anomaly detection with semantic similarity," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2438–2439.
- [6] L. Yang, J. Chen, S. Gao, Z. Gong, H. Zhang, Y. Kang, and H. Li, "Try with simpler-an evaluation of improved principal component analysis in log-based anomaly detection," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 5, pp. 1–27, 2024.
- [7] B. Yu, J. Yao, Q. Fu, Z. Zhong, H. Xie, Y. Wu, Y. Ma, and P. He, "Deep learning or classical machine learning? an empirical study on log-based anomaly detection," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.

- [8] S. Yu, Y. Wu, Y. Li, and P. He, "Unlocking the power of numbers: Log compression via numeric token parsing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 919–930.
- [9] X. Li, H. Zhang, V.-H. Le, and P. Chen, "Logshrink: Effective log compression by leveraging commonality and variability of log data," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–12.
- [10] J. Liu, J. Zhu, S. He, P. He, Z. Zheng, and M. R. Lyu, "Logzip: Extracting hidden structures via iterative clustering for log compression," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 863–873.
- [11] D. Roy, X. Zhang, R. Bhave, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, "Exploring llm-based agents for root cause analysis," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 208–219.
- [12] S. Shan, Y. Huo, Y. Su, Y. Li, D. Li, and Z. Zheng, "Face it yourselves: An llm-based two-stage strategy to localize configuration errors via logs," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 13–25.
- [13] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2438–2449.
- [14] Y. Xiao, V.-H. Le, and H. Zhang, "Free: Towards more practical log parsing with large language models," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 153–165.
- [15] J. Xu, R. Yang, Y. Huo, C. Zhang, and P. He, "Divlog: Log parsing with prompt enhanced in-context learning," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.
- [16] Y. Huo, Y. Su, C. Lee, and M. R. Lyu, "Semparser: A semantic parser for log analytics," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 881–893.
- [17] Z. A. Khan, D. Shin, D. Bianculli, and L. Briand, "Guidelines for assessing the accuracy of log message template identification techniques," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1095–1106.
- [18] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, "Tools and benchmarks for automated log parsing," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2019, pp. 121–130.
- [19] Z. Jiang, J. Liu, Z. Chen, Y. Li, J. Huang, Y. Huo, P. He, J. Gu, and M. R. Lyu, "Lilac: Log parsing using llms with adaptive parsing cache," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 137–160, 2024.
- [20] V.-H. Le and H. Zhang, "Log parsing: How far can chatgpt go?" in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 1699–1704.
- [21] Z. Ma, D. J. Kim, and T.-H. Chen, "Librelog: Accurate and efficient unsupervised log parsing using open-source large language models," *arXiv preprint arXiv:2408.01585*, 2024.
- [22] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [23] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," *arXiv preprint arXiv:2409.02977*, 2024.
- [24] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," *arXiv preprint arXiv:2402.01680*, 2024.
- [25] F. Wu, S. Wu, Y. Cao, and C. Xiao, "Wipi: A new web threat for llm-driven web agents," *arXiv preprint arXiv:2402.16965*, 2024.
- [26] A. Bohra, M. Saroyan, D. Melkozerov, V. Karufanyan, G. Maher, P. Weinberger, A. Harutyunyan, and G. Campagna, "Weblists: Extracting structured information from complex interactive websites using executable llm agents," *arXiv preprint arXiv:2504.12682*, 2025.
- [27] Anthropic. (2025) Claude 3.7 sonnet. [Online]. Available: <https://www.anthropic.com/claude/sonnet#customer-stories>
- [28] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for ai-driven log analytics," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2023, pp. 355–366.
- [29] S. Yu, P. He, N. Chen, and Y. Wu, "Brain: Log parsing with bidirectional parallel tree," *IEEE Transactions on Services Computing*, vol. 16, no. 5, pp. 3224–3237, 2023.
- [30] Y. Liu, X. Zhang, S. He, H. Zhang, L. Li, Y. Kang, Y. Xu, M. Ma, Q. Lin, Y. Dang *et al.*, "Uniparser: A unified log parser for heterogeneous log data," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 1893–1901.
- [31] X. Wang, X. Zhang, L. Li, S. He, H. Zhang, Y. Liu, L. Zheng, Y. Kang, Q. Lin, Y. Dang *et al.*, "Spine: a scalable log parser with feedback guidance," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 1198–1208.
- [32] Z. Jiang, J. Liu, J. Huang, Y. Li, Y. Huo, J. Gu, Z. Chen, J. Zhu, and M. R. Lyu, "A large-scale evaluation for log parsing techniques: How far are we?" in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 223–234.
- [33] A. Vervaet, M. Callau-Zori, Y. Chabchoub, and R. Chiky, "Online log parsing using evolving research tree," *Knowledge and Information Systems*, vol. 66, no. 2, pp. 1231–1255, 2024.
- [34] Z. M. Jiang, A. E. Hassan, P. Flora, and G. Hamann, "Abstracting execution logs to execution events for enterprise applications (short paper)," in *2008 The Eighth International Conference on Quality Software*. IEEE, 2008, pp. 181–186.
- [35] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *2017 IEEE international conference on web services (ICWS)*. IEEE, 2017, pp. 33–40.
- [36] R. Vaarandi, "A data clustering algorithm for mining patterns from event logs," in *Proceedings of the 3rd IEEE Workshop on IP Operations & Management (IPOM 2003)(IEEE Cat. No. 03EX764)*. Ieee, 2003, pp. 119–126.
- [37] R. Vaarandi and M. Pihelgas, "Logcluster-a data clustering and pattern mining algorithm for event logs," in *2015 11th International conference on network and service management (CNSM)*. IEEE, 2015, pp. 1–7.
- [38] Q. Fu, J.-G. Lou, Y. Wang, and J. Li, "Execution anomaly detection in distributed systems through unstructured log analysis," in *2009 ninth IEEE international conference on data mining*. IEEE, 2009, pp. 149–158.
- [39] L. Tang, T. Li, and C.-S. Perng, "Logsig: Generating system events from raw textual logs," in *Proceedings of the 20th ACM international conference on Information and knowledge management*, 2011, pp. 785–794.
- [40] H. Hamooni, B. Debnath, J. Xu, H. Zhang, G. Jiang, and A. Mueen, "Logmine: Fast pattern recognition for log analytics," in *Proceedings of the 25th ACM international on conference on information and knowledge management*, 2016, pp. 1573–1582.
- [41] M. Mizutani, "Incremental mining of system log format," in *2013 IEEE International Conference on Services Computing*. IEEE, 2013, pp. 595–602.
- [42] K. Shima, "Length matters: Clustering system log messages using length of words. arxiv 2016," *arXiv preprint arXiv:1611.03213*.
- [43] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "Towards automated log parsing for large-scale log data analysis," *IEEE Transactions on Dependable and Secure Computing*, vol. 15, no. 6, pp. 931–944, 2017.
- [44] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [45] X. Yu, S. Nong, D. He, W. Zheng, T. Ma, N. Liu, J. Li, and G. Xie, "Loggenius: An unsupervised log parsing framework with zero-shot prompt engineering," in *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024, pp. 1321–1328.
- [46] W. Zhang, H. Guo, A. Le, J. Yang, J. Liu, and Z. Li, "Lemur: Log parsing with entropy sampling and chain-of-thought merging," *arXiv preprint arXiv:2402.18205*, 2024.